Homework 4

Intel oneMKL supports the product of a sparse matrix and a dense matrix and stores the result as a dense matrix [1]. The function is

```c
sparse_status_t mkl_sparse_s_mm (
        const sparse_operation_t operation,
        const float alpha,
        const sparse_matrix_t A, const struct matrix_descr descr,
        const sparse_layout_t layout,
        const float *B, const MKL_INT columns, const MKL_INT ldb,
        const float beta, float *C, const MKL_INT ldc);
```

In this problem, you are required to implement a function in **C language** to support the multiplication

$$C = AB$$

where A, B and C are a sparse matrix, a dense matrix and the target dense matrix, respectively. Then, you will compare the performance of your implementation with the implementation from Intel oneMKL.

(**Note:** You may use 217's machines if you encounter difficulties installing Intel oneMKL, where the library is already installed.)

Requirements

(1) **Implement the compressed column/row matrix format** and the **function for multiplication**. Include the code in your report. Your function should be able to support A with both CSC and CSR formats. Your storage scheme for the dense matrix can be either column or row-major layout.

(2) **Benchmark cases:** Use the following cases when generating random matrices and benchmarking your implementation and Intel oneMKL:

- **Shapes for A:** $\in \{(n, n), (4n, n), (n, 4n)\}$, where $n \in \{2048, 32768\}$
- **Number of columns of B:** $k \in \{64, 512\}$
- **Sparsity control:**
  - For both CSR and CSC formats, the total number of non-zeros (NNZ) should be $32\times$m (where m is the number of rows of A).

(You are expected to use these settings, but if your hardware constraints require changes, please state exactly why you modified them.)

(3) Perform complexity analysis for your multiplication. Try to optimize your speed by implementing **OpenMP** parallelization for either CSR or CSC. Clearly state which format you parallelized and why.

(4) Benchmark your implementation and compare it with Intel oneMKL library, and explain if and why the results matched your expectations.

[1] https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl-documentation.html